FACULDADE DE INFORMÁTICA E ADMINISTRAÇÃO PAULISTA
ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

ARCGARDEN

Gamificação da sustentabilidade

DOCUMENTAÇÃO TÉCNICA • SPRINT 4

Equipe: ARCEUS

Franco Jared Marquina Blas (RM 572901)
Igor Mateus da Silva (RM 571751)
Marcela Batista Teixeira (RM 574069)
Matheus Pereira Vidal (RM 573327)
Sam Giulian de Souza Gibelli Altoe (RM 570332)

São Paulo • 2026

SUMÁRIO

1 :…….. Descrição do projeto
2 :…….. Arquitetura e tecnologias
3 :…….. Funcionalidades implementadas
4 :…….. Tabela de endpoints
5 :…….. Modelo de dados (MER)
6 :…….. Diagrama de classes
7 :…….. Procedimentos para execução
8 :…….. Evidências e pontos a complementar

1. DESCRIÇÃO DO PROJETO

1.1 Objetivo do projeto

O ArcGarden é uma solução de software baseada em gamificação que combina o gerenciamento de jardins virtuais com a realização de missões relacionadas à sustentabilidade. O sistema busca incentivar hábitos sustentáveis por meio de uma experiência interativa na qual os jogadores podem manter jardins personalizados, cultivar plantas com diferentes características e acompanhar seu desempenho em um ranking.

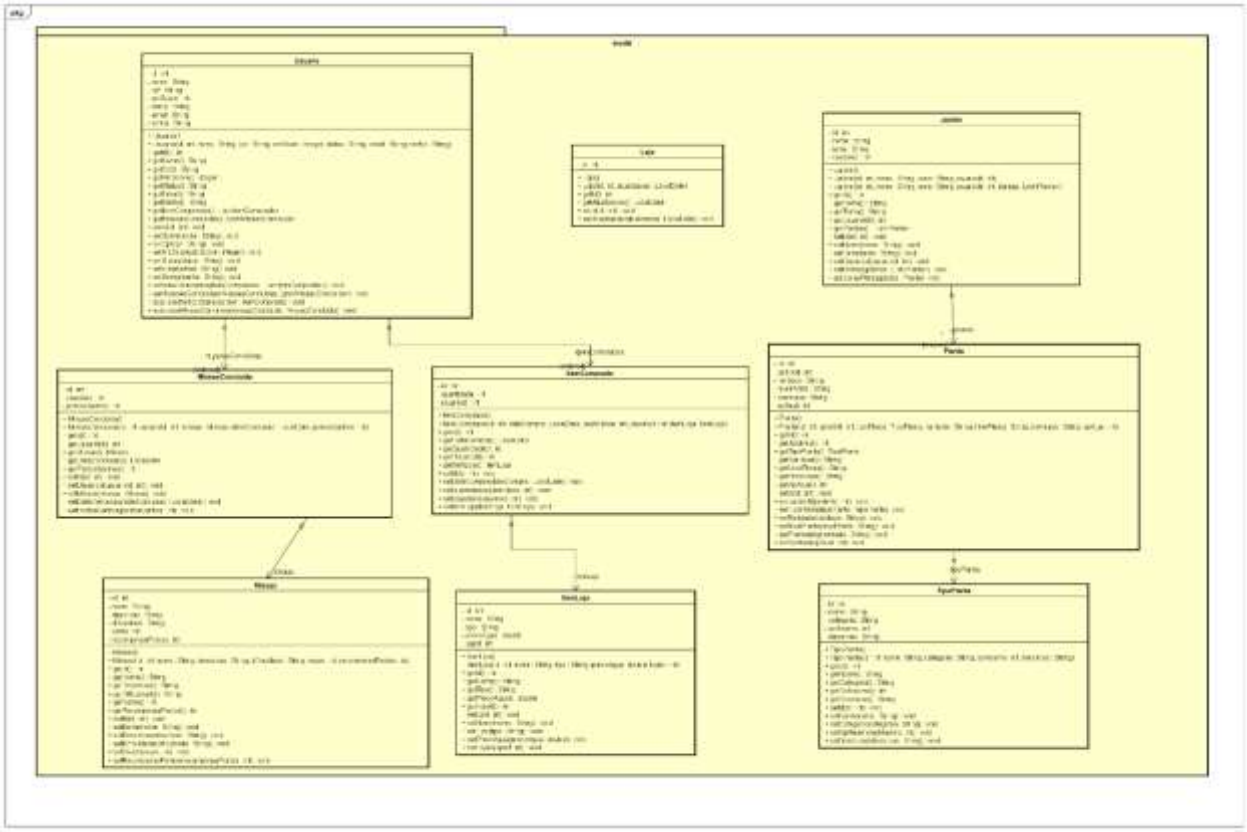
Na evolução documentada para a Sprint 4, o projeto Java passou a disponibilizar uma API REST construída com Spring Boot. Essa API fornece operações para administrar usuários, jardins, plantas, missões, lojas e itens, além de consultar o ranking e registrar missões concluídas e compras.

1.2 Escopo funcional observado

- Cadastro, consulta e atualização de usuários, incluindo dados de perfil e pontuação arcScore.
- Consulta de ranking ordenado pela pontuação arcScore.
- Criação, consulta, atualização e exclusão de jardins, com filtros por tema e usuário.
- Inclusão de plantas em jardins e consulta das plantas associadas com seus tipos.
- Gerenciamento de missões e tipos de plantas, com operações de cadastro, consulta, atualização e exclusão.
- Gerenciamento de lojas e itens disponibilizados para compra.
- Registro de conclusão de missões e de compras de itens associados aos usuários.
- Consulta do histórico de missões concluídas e de itens comprados ao consultar um usuário.

2. ARQUITETURA E TECNOLOGIAS

A aplicação está organizada em camadas, separando a exposição dos recursos HTTP, a conversão dos dados recebidos e enviados, as regras de negócio e o acesso ao banco de dados. O projeto utiliza classes Java simples para representar o domínio e realiza as operações SQL diretamente nos repositórios.



2.1 Camadas e responsabilidades

Camada / pacote	Responsabilidade observada
controller	Classes Resource anotadas com @RestController; recebem requisições HTTP, chamam os serviços e montam respostas.
controller.dto	Records de request/response e classes Mapper para converter DTOs em modelos e vice-versa.
service	Reúne operações do sistema, validações de campos e chamadas aos repositórios.
repository	Executa comandos SQL por PreparedStatement, utilizando DataSource e DataSourceUtils para obter/liberar conexões.
model	Classes do domínio: Usuario, Jardim, Planta, TipoPlanta, Missao, Loja e entidades de histórico/compra.
config	WebConfig configura CORS global por WebMvcConfigurer; origem * e métodos GET, POST, PUT, DELETE e OPTIONS.
exceptions	ErrorResponse padroniza os dados básicos das respostas de erro.

2.2 Tecnologias identificadas

Tecnologia / ferramenta	Uso identificado
Java 17	Versão configurada no pom.xml e na imagem de execução do Dockerfile.
Spring Boot 4.1.1	Inicialização e configuração da aplicação.
Spring Web MVC	Construção dos recursos REST e tratamento das requisições HTTP.

Spring JDBC + Oracle ojdbc17	Conexão e comunicação com o banco Oracle via JDBC.
Maven	Gerenciamento de dependências e empacotamento do projeto.
Docker	Dockerfile com etapa de build Maven e execução em imagem Java 17 JRE.

2.3 Comunicação e erros

As classes Resource retornam ResponseEntity e utilizam os códigos HTTP 201 (Created), 200 (OK), 204 (No Content), 400 (Bad Request) e 404 (Not Found), conforme cada operação. As respostas de erro usam o record ErrorResponse, composto por timestamp, status, message e path.

3. FUNCIONALIDADES IMPLEMENTADAS

3.1 Recursos disponíveis

Recurso	Funcionalidade observada
Usuários	Cadastrar, listar, consultar por ID e atualizar. O retorno da consulta inclui listas de compras e missões concluídas.
Ranking	Listar usuários em ordem decrescente de arcScore e consultar um usuário por posição.
Jardins	Criar, listar, consultar por ID, atualizar e excluir. A listagem permite filtro por tema ou usuarioId.
Plantas	Adicionar uma planta a um jardim existente. A consulta do jardim carrega as plantas e os dados de TipoPlanta associados.
Missões	Cadastrar, listar, consultar, atualizar e excluir missões.
Tipos de plantas	Cadastrar, listar, consultar, atualizar e excluir tipos; a listagem aceita filtro por categoria.
Lojas	Cadastrar, listar, consultar, atualizar e excluir lojas.
Itens da loja	Cadastrar, listar, consultar, atualizar e excluir itens; a listagem aceita filtro por tipo.
Históricos	Registrar missão concluída e compra de item para um usuário; recuperar os registros ao consultar o usuário.

3.2 Regras e validações encontradas

Entidade / operação	Validações implementadas no Service
Jardim	Nome e tema obrigatórios; usuarioId deve ser maior que zero; ao adicionar planta, o tipoId deve ser válido numericamente.
Missão	Nome obrigatório; dificuldade deve ser F, M ou D; vezes maior que zero; recompensa entre 0 e 100.
Tipo de planta	Nome e categoria obrigatórios; xpMaximo maior que zero.
Item da loja	Nome e tipo obrigatórios; preço em água não negativo; lojaId maior que zero.
Loja	Data de atualização obrigatória.
Compra de item	Quantidade deve ser maior que zero; o parâmetro quantidade assume 1 quando não informado na rota.
Usuário	O Service não apresenta validação geral de campos equivalente às demais entidades no código recebido.

3.3 Observações sobre as regras de negócio

- O registro de missão concluída grava a identificação do usuário, da missão e a data atual no banco. O histórico carregado calcula pontosGanhos com base em recompensaPontos da missão, mas não há atualização de arc_score na operação observada.
- O registro de compra grava a data, quantidade, usuário e item da loja. Não foi identificado débito de saldo ou consumo de arcScore nessa operação.

4. TABELA DE ENDPOINTS — API REST

A URL-base local configurada para os exemplos é http://localhost:8080. Os caminhos abaixo são relativos à raiz da aplicação e foram extraídos das anotações de mapeamento dos Resources.

4.1 Usuários e ranking

Método	URI	Finalidade	Resposta indicada no código
POST	/api/v1/usuarios	Cadastra usuário.	201; erro 400
GET	/api/v1/usuarios	Lista usuários, compras e missões concluídas.	200; erro 404
GET	/api/v1/usuarios/{id}	Consulta usuário por ID e respectivos históricos.	200; 404
PUT	/api/v1/usuarios/{id}	Atualiza dados do usuário.	204; erro 400
POST	/api/v1/usuarios/{usuarioId}/missoes/{missaoId}	Registra conclusão de missão para o usuário.	201; erro 400
POST	/api/v1/usuarios/{usuarioId}/itens/{itemId}?quantidade=1	Registra compra; quantidade é opcional e padrão 1.	201; erro 400
GET	/api/v1/ranking/mensal	Lista usuários ordenados por arcScore decrescente.	200; erro 404
GET	/api/v1/ranking/mensal/{posicao}	Consulta usuário em uma posição do ranking.	200; 404

4.2 Jardins e plantas

Método	URI	Finalidade	Resposta indicada no código
POST	/api/v1/jardins	Cria jardim.	201; 400
GET	/api/v1/jardins	Lista jardins; aceita tema e/ou usuarioId como filtros.	200; 404
GET	/api/v1/jardins/{id}	Consulta jardim e plantas associadas.	200; 404
PUT	/api/v1/jardins/{id}	Atualiza nome e tema do jardim.	204; 400
DELETE	/api/v1/jardins/{id}	Exclui jardim.	204; 400
POST	/api/v1/jardins/{id}/plantas	Adiciona planta ao jardim.	201; 400

4.3 Missões

Método	URI	Finalidade	Resposta indicada no código
POST	/api/v1/missoes	Cadastra missão.	201; 400
GET	/api/v1/missoes	Lista missões.	200; 404
GET	/api/v1/missoes/{id}	Consulta missão por ID.	200; 404
PUT	/api/v1/missoes/{id}	Atualiza missão.	204; 400
DELETE	/api/v1/missoes/{id}	Exclui missão.	204; 400

4.4 Tipos de plantas

Método	URI	Finalidade	Resposta indicada no código

POST	/api/v1/tipos-plantas	Cadastra tipo de planta.	201; 400
GET	/api/v1/tipos-plantas	Lista tipos; aceita categoria como filtro opcional.	200; 404
GET	/api/v1/tipos-plantas/{id}	Consulta tipo por ID.	200; 404
PUT	/api/v1/tipos-plantas/{id}	Atualiza tipo de planta.	204; 400
DELETE	/api/v1/tipos-plantas/{id}	Exclui tipo de planta.	204; 400

4.5 Lojas

Método	**URI**	**Finalidade**	**Resposta indicada no código**
POST	/api/v1/lojas	Cadastra loja.	201; 400
GET	/api/v1/lojas	Lista lojas.	200; 404
GET	/api/v1/lojas/{id}	Consulta loja por ID.	200; 404
PUT	/api/v1/lojas/{id}	Atualiza loja.	204; 400
DELETE	/api/v1/lojas/{id}	Exclui loja.	204; 400

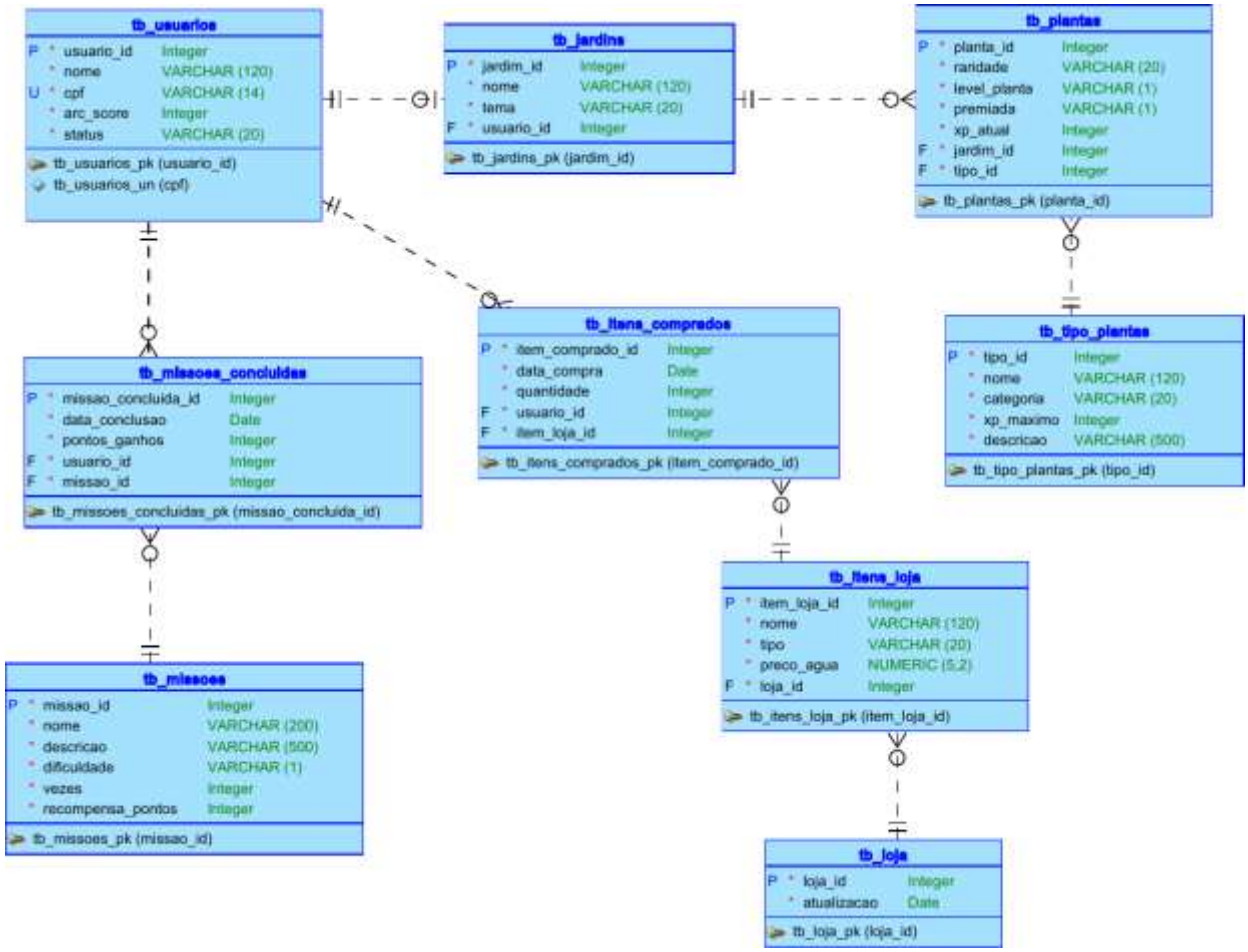
4.6 Itens da loja

Método	**URI**	**Finalidade**	**Resposta indicada no código**
POST	/api/v1/itens-loja	Cadastra item da loja.	201; 400
GET	/api/v1/itens-loja	Lista itens; aceita tipo como filtro opcional.	200; 404
GET	/api/v1/itens-loja/{id}	Consulta item por ID.	200; 404
PUT	/api/v1/itens-loja/{id}	Atualiza item da loja.	204; 400
DELETE	/api/v1/itens-loja/{id}	Exclui item da loja.	204; 400

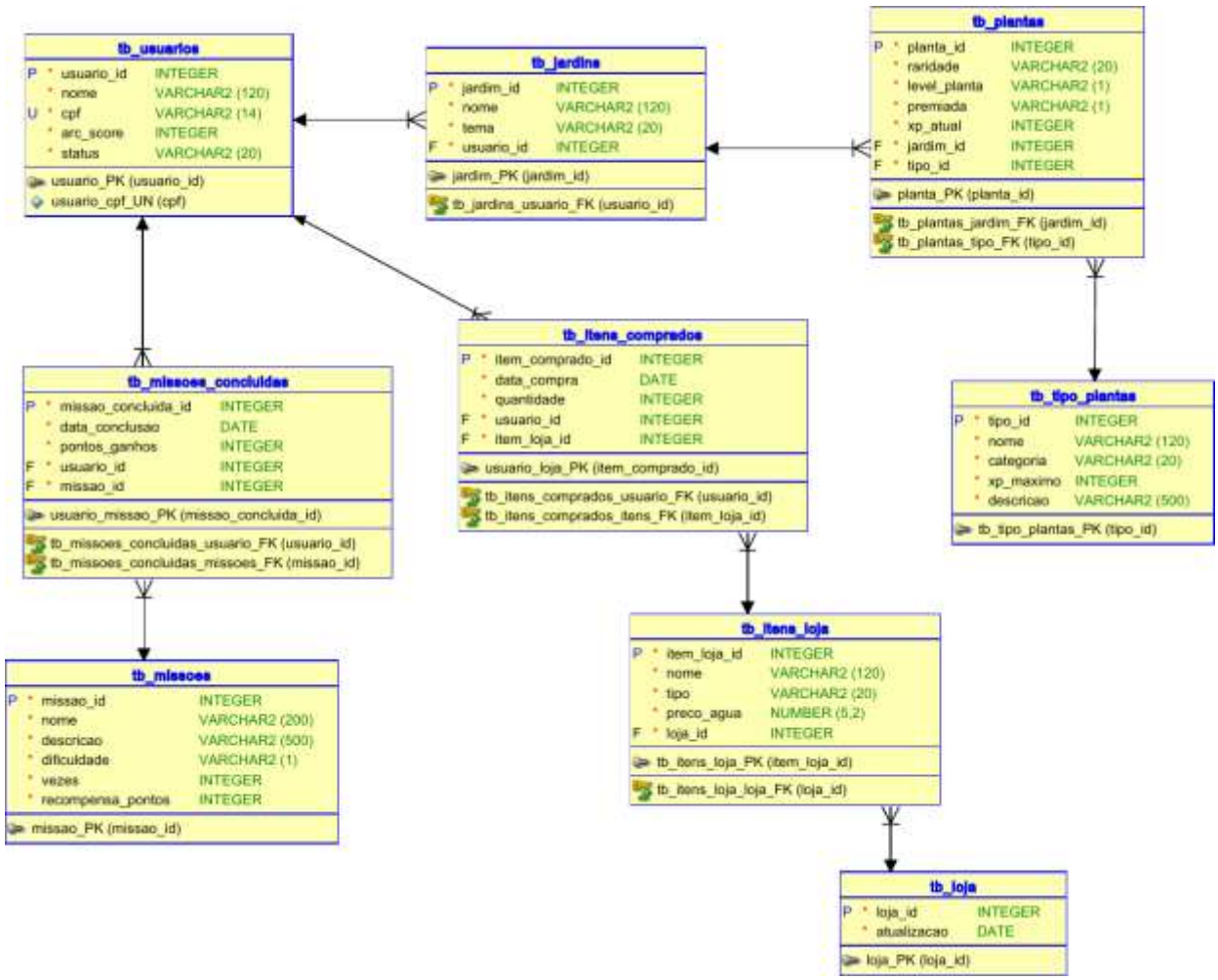
5. Modelo de Entidade-Relacionamento (MER)

O banco de dados do Arc Garden é composto por nove entidades: tb_usuarios, tb_jardins, tb_plantas, tb_tipo_plantas, tb_missoes, tb_missoes_concluidas, tb_itens_loja, tb_itens_comprados e tb_loja. Um usuário pode possuir vários jardins, e cada jardim pode conter várias plantas, cada uma associada a um tipo de planta do catálogo. As tabelas tb_missoes_concluidas e tb_itens_comprados funcionam como entidades associativas, registrando, respectivamente, quais missões cada usuário concluiu e quais itens da loja cada usuário comprou.

Modelo Lógico



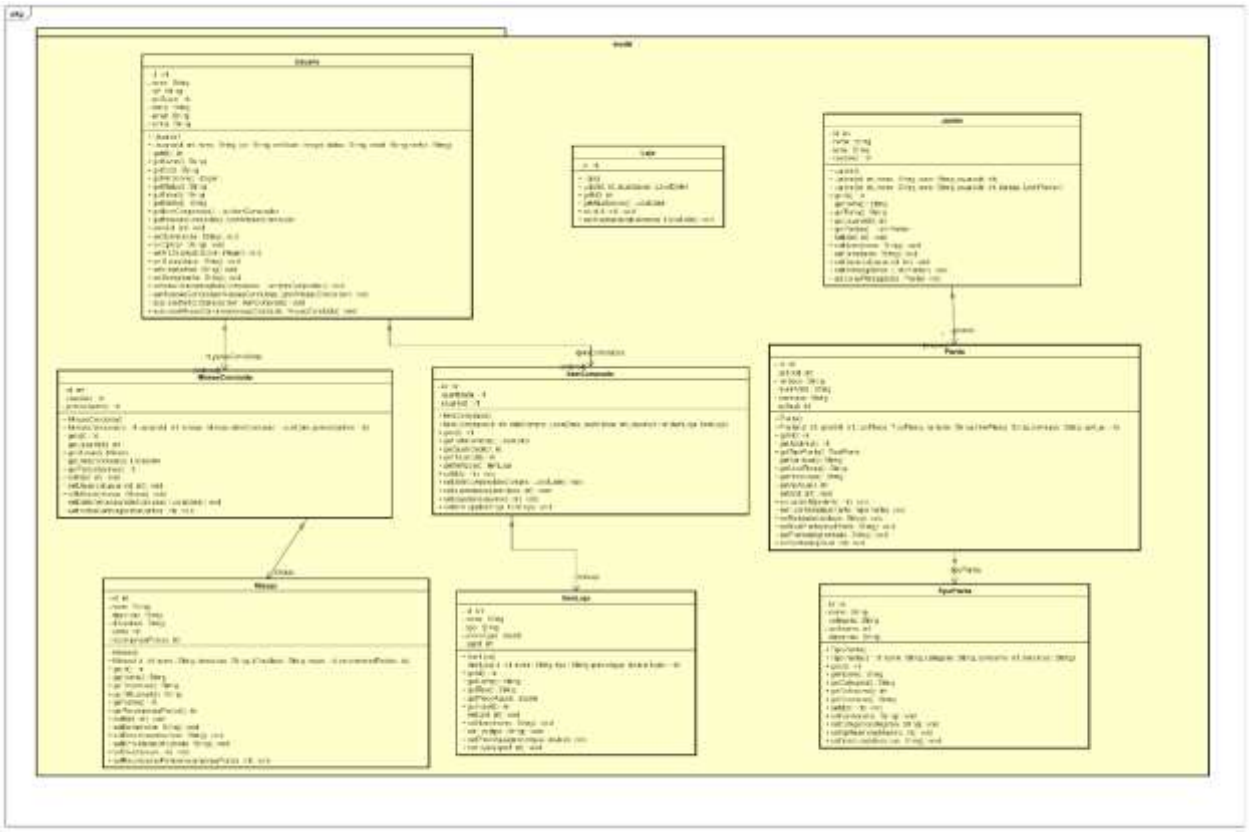
Modelo Físico / Relacional (Oracle)



Modelo físico com tipos de dado Oracle (VARCHAR2/NUMBER) e nomes de chaves primárias e estrangeiras.

6. Diagrama de Classes

O diagrama abaixo representa a camada model do projeto Java tal como implementada até esta sprint (pacote br.com.fiap.arcgarden.model), com atributos, métodos e multiplicidades dos relacionamentos. Todos os relacionamentos são de agregação (losango vazio): cada classe "parte" (Missao, ItemLoja, Planta) é uma entidade de catálogo que existe de forma independente e é apenas referenciada pela classe "todo".



7. PROCEDIMENTOS PARA EXECUÇÃO

7.1 Ferramentas e requisitos

Item	Versão / observação
JDK	Java 17.
Build	Maven; o Dockerfile usa Maven 3.8.5 na etapa de compilação.
Framework	Spring Boot 4.1.1, conforme pom.xml.
Banco	Oracle Database acessível pela URL JDBC configurada para o ambiente.
Porta local	8080, conforme application.properties.

7.2 Execução local com Maven

1. Extraia o ZIP e abra a pasta ArcGarden-Java na IDE ou no terminal.

2. Configure a conexão com o Oracle para o ambiente utilizado. Defina URL, usuário e senha por variáveis de ambiente ou por um arquivo local não versionado.

3. No terminal, dentro da pasta que contém pom.xml, execute: mvn clean package.

4. Inicie a aplicação com: mvn spring-boot:run.

5. A API ficará disponível localmente na porta 8080. Utilize http://localhost:8080/api/v1 como raiz dos endpoints.

7.3 Variáveis de ambiente recomendadas

Variável	Finalidade
SPRING_DATASOURCE_URL	URL JDBC do banco Oracle utilizado.
SPRING_DATASOURCE_USERNAME	Usuário de conexão com o banco.
SPRING_DATASOURCE_PASSWORD	Senha de conexão com o banco.
SERVER_PORT	Porta HTTP da aplicação; padrão 8080.

7.4 Execução com Docker

O projeto possui Dockerfile com build Maven e runtime Java 17. Na pasta do projeto, a imagem pode ser construída com:

```
docker build -t arcgarden-api .
```

Após configurar as variáveis de ambiente necessárias ao container e garantir acesso ao Oracle, a execução pode ser feita mapeando a porta 8080 do container para a máquina local.

8. EVIDÊNCIAS E PONTOS A COMPLEMENTAR

Os arquivos recebidos permitem documentar a estrutura do backend Java e as rotas definidas nos controllers. Alguns entregáveis previstos no enunciado da Sprint 4 não estão presentes no material recebido ou dependem de validação em ambiente real.

Item	**Situação observada nos arquivos**	**Ação para fechar a documentação final**
Capturas de tela do sistema	Não foram enviados front-end nem prints de telas.	Inserir capturas reais da aplicação em funcionamento, com legenda e breve explicação.
Deploy da API	README contém o campo do link do Render ainda como “todo”.	Publicar a API e substituir o marcador pelo endereço público; validar acesso externo.
MER oficial / modelo físico	modelo.sql está vazio; relações foram inferidas dos repositórios.	Anexar o MER/diagrama físico oficial e conferir PK, FK, tipos e constraints no Oracle.
Teste da API integrada	O ZIP não contém evidência de execução contra Oracle ou cliente front-end.	Executar e registrar testes dos endpoints, incluindo operações CRUD e integração com o front-end.
Exclusão de usuário	Não há DELETE para UsuarioResource/UsuarioService/UsuarioRepository.	Implementar a operação, caso a equipe/professor exija CRUD completo para usuários.
Ranking mensal	A consulta ordena arc_score, sem filtro de período.	Implementar critério mensal real ou alterar a descrição/rota para refletir o ranking global atual.
Pontuação por missão	Conclusão grava histórico, mas não atualiza arc_score.	Implementar a regra de crédito da recompensa, caso essa seja a regra esperada do produto.
CorsFilter	O projeto usa WebConfig/WebMvcConfigurer para CORS; não há classe chamada CorsFilter.	Confirmar se a configuração atual atende ao requisito da disciplina ou criar a classe exigida literalmente.
Build e testes automatizados	Existe teste padrão contextLoads; não foi possível executar Maven neste ambiente porque o comando mvn não está instalado.	Executar mvn clean package e validar operações com o banco antes de afirmar que a API está funcional.

8.1 Conclusão

Na Sprint 4, o ArcGarden apresenta uma evolução estrutural em relação à documentação da Sprint 3: o backend Java está organizado como API REST em Spring Boot, possui recursos HTTP para os principais elementos do domínio e utiliza repositórios JDBC para persistência no Oracle. A documentação registra a arquitetura e as rotas identificadas no código e explicita os itens que ainda precisam de evidência, ajuste ou validação para caracterizar a entrega final como integrada e publicada.